

112. Path Sum

PLATFORM

Platform: LeetCode

DIFFICULTY

EASY

DATE

Solved: July 31, 2026

Problem Summary

Given the root of a binary tree and a targetSum, return true if there exists a root-to-leaf path where all node values sum to targetSum.

Approach

Recursive DFS — subtract current node's value from target at each step. At a leaf, check if remaining target equals leaf's value.

Algo

1. If root == null return false.
2. If leaf node → return targetSum == root.val.
3. Recurse: hasPathSum(left, target - root.val) || hasPathSum(right, target - root.val).

Time Complexity & Space Complexity

O(n) — visits every node in worst case.

O(h) — recursion stack.

Key Insights

- Subtracting instead of adding — by the time you reach a leaf, remaining target should equal leaf value.
- Leaf check: root.left == null && root.right == null — both children null.
- || short circuits — if left path found, right path never explored.
- Must reach a **leaf** — not just any node where sum matches.

Observations

- targetSum == root.val not targetSum - root.val == 0 — same thing, but former is cleaner.
- Works with negative values — subtraction handles them naturally.
- No global variable needed — pure recursive return value.

112. Path Sum Solved

Easy Topics Companies

Given the root of a binary tree and an integer targetSum, return true if the tree has a root-to-leaf path such that adding up all the values along the path equals targetSum.

A leaf is a node with no children.

Example 1:

```
graph TD
    5((5)) --- 4((4))
    5((5)) --- 8((8))
    4((4)) --- 11((11))
    4((4)) --- 13((13))
    11((11)) --- 7((7))
    11((11)) --- 2((2))
    8((8)) --- 4((4))
    4((4)) --- 1((1))
```

Input: root = [5,4,8,11,null,13,4,7,2,null,null,null,1], targetSum = 22
Output: true
Explanation: The root-to-leaf path with the target sum is shown.

Example 2:

```
graph TD
    1((1))
```

Runtime: 0 ms Beats 100.00%
Memory: 44.70 MB Beats 93.04%

Code:

```
class Solution {
    public boolean hasPathSum(TreeNode root, int targetSum) {
        if (root == null) return false;
        if (root.left == null && root.right == null)
            return targetSum == root.val;
        return hasPathSum(root.left, targetSum - root.val) ||
            hasPathSum(root.right, targetSum - root.val);
    }
}
```

Testcase: Accepted Runtime: 0 ms

Case 1: Input: root = [5,4,8,11,null,13,4,7,2,null,null,null,1], targetSum = 22

